

NOMBRE DE RACINES RÉELLES D'UN POLYNÔME RÉEL DANS UN INTERVALLE : THÉORÈME DE **STURM**

FOLLY-DOGBA Folly Romuald

folly.folly-dogba@etu.unige.ch

Université de Genève

Juin 2024

Table des matières

1 - Introduction

2 - Le polynôme et son vecteur

2.1 - Polynôme, degré et représentation vectorielle

2.2 - Dérivée d'une fonction polynôme

2.3 - Représentation dans Python

3 - Division polynomiale

3.1 - La division polynomiale, c'est quoi ?

3.2 - Implémentation de la procédure

4 - Théorème de Sturm

4.1 - Énoncé du théorème

4.2 - Implémentation du théorème

5 - Conclusion

1 - Introduction

Dans le cadre de ce projet, il est question de trouver le nombre de solutions réelles que possède un polynôme réel sur un intervalle donné. Pour cela, nous utiliserons une forme vectorielle pour représenter les polynômes afin de mieux les manipuler en **Python**. Il est question ici d'utiliser le **théorème de Sturm**. Nous ferons intervenir pour cela des divisions polynomiales successives entre polynômes et les notions de dérivées de polynômes.

2 - Le polynôme et son vecteur

1 - Introduction

2 - Le polynôme et son vecteur

2.1 - Polynôme, degré et représentation vectorielle

2.2 - Dérivée d'une fonction polynôme

2.3 - Représentation dans Python

3 - Division polynomiale

4 - Théorème de Sturm

5 - Conclusion

2.1 - Polynôme, degré et représentation vectorielle

Un polynôme réel P **de degré** n est une fonction :

$$\begin{aligned} P : \mathbb{R} &\rightarrow \mathbb{R} \\ x &\mapsto P(x) = a_0 + a_1x + a_2x^2 + \cdots + a_nx^n \end{aligned}$$

Cette représentation peut être associée à un vecteur de taille n $v = (a_0, a_1, \cdots, a_n)$, où chaque composante du vecteur correspond à un coefficient du polynôme dans la base $\{1, x, x^2, \cdots, x^n\}$.

En prenant v comme le polynôme vu en composantes dans cette base, nous pouvons manipuler les polynômes de manière plus efficace en utilisant les opérations vectorielles. Cela permet d'appliquer des algorithmes d'analyse et de manipulation de polynômes avec des outils numériques tels que **Python**.

2.1 - Polynôme, degré et représentation vectorielle (suite)

Exemple : Le polynôme $P(x) = -3 + 7x^2 - 2x^7$ est de **degré 7** et sera représenté par $v = (-3, 0, 7, 0, 0, 0, 0, -2)$.

Le vecteur v contient **8** éléments qui sont les coefficients du polynôme P .

Je ne suis pas d'accord

v doit normalement être égal à $(-3, 7, -2)$. Ou bien ?

Techniquement, oui. Mais dans le contexte de la représentation donnée dans ce projet, le polynôme $P(x) = -3 + 7x^2 - 2x^7$ doit être vu comme :

$$P(x) = -3 + 0x + 7x^2 + 0x^3 + 0x^4 + 0x^5 + 0x^6 - 2x^7$$

2.2 - Dérivée d'une fonction polynôme

La fonction dérivée d'un polynôme réel P **de degré** n est elle aussi un polynôme réel P' mais de degré $n - 1$ définie par :

$$\begin{aligned} P' : \mathbb{R} &\rightarrow \mathbb{R} \\ x &\mapsto P'(x) = a_1 + 2 * a_2x + \dots + n * a_nx^{n-1} \end{aligned}$$

La dérivée d'un polynôme P , notée P' , représente le taux de variation de P par rapport à sa variable indépendante, souvent notée ici x . Elle est un outil puissant utilisé pour analyser et comprendre les propriétés et le comportement des fonctions général et polynômes en particulier dans ce cadre de ce projet.

Exemple: Pour

$$P(x) = -6 + 2x - 3x^2 + 7x^4, \text{ on a } P'(x) = 2 - 6x + 28x^3$$

2.3 - Représentation dans Python

Ci-dessus dans la sous section 2.1 nous avons vu comment représenter un polynôme sous forme de vecteur.

Pour mieux gérer et manipuler les données, nous allons représenter un polynôme (et donc une dérivée aussi) en Python par une **classe** possédant des attributs et des méthodes.

Sur la page suivante, nous donnons le code python de définition de la classe et de son constructeur. Le reste du code de la classe est représenté dans le fichier .ipynb associé à ce projet.

Code Python de la classe polynôme P et son constructeur

```
import numpy as np

class P:
    """
    > Cette classe représente d'un polynôme quelconque donné
    - Elle prend en arguments les coefficients (a0, a1, ..., aN)
      sous forme de tuple, de liste ou de tableau numpy
    - Elle prend aussi en argument un nom. Par défaut, P
    """
    def __init__(self, coefs: np.ndarray|tuple|list, name = "P"):
        # nom du polynôme
        self.name = name
        # suppression d'ordre ligne-colonne avec .ravel()
        # et affectation des coefficients
        self.coefs = np.array(coefs, dtype=np.float64).ravel()
        # détermination du degré du polynôme
        self.degree = self.coefs.shape[0] - 1
```

Exemple d'instanciation et d'affichage

```
# Utilisation de la classe
f = P([2, 1, 1], name="f")
g = P([0, 1, 3, 3, 1/8], name="g")

# affichage en ligne de commande
print(f)
print(g)

# affichage avec Sympy
f.beautiful_display()
g.beautiful_display()
```

✓ 0.0s

$$f(X) = 2.0 + X + X^2$$

$$g(X) = X + 3.0X^2 + 3.0X^3 + 0.125X^4$$

$$f(x) = 1.0x^2 + 1.0x + 2.0$$

$$g(x) = 0.125x^4 + 3.0x^3 + 3.0x^2 + 1.0x$$

Division polynomiale

1 - Introduction

2 - Le polynôme et son vecteur

3 - Division polynomiale

3.1 - La division polynomiale, c'est quoi ?

3.2 - Implémentation de la procédure

4 - Théorème de Sturm

5 - Conclusion

3.1 - La division polynomiale, c'est quoi ?

La division polynomiale est une technique utilisée pour diviser un polynôme $P(x)$ par un autre polynôme $D(x)$. Le résultat de cette division est un quotient $Q(x)$ et un reste $R(x)$ tels que :

$$P(x) = D(x) \cdot Q(x) + R(x)$$

où le degré de $R(x)$ est strictement inférieur au degré de $D(x)$. Cette méthode est analogue à la division des nombres entiers. Les étapes de la division polynomiale sont les suivantes :

1. Diviser le terme de plus haut degré de $P(x)$ par le terme de plus haut degré de $D(x)$ pour obtenir le premier terme de $Q(x)$.
2. Multiplier $D(x)$ par ce premier terme de $Q(x)$ et soustraire le résultat de $P(x)$.
3. Répéter les étapes précédentes avec le polynôme obtenu jusqu'à ce que le degré du reste soit inférieur au degré de $D(x)$.

3.1 - La division polynomiale, c'est quoi ? (Exemple)

Considérons les polynômes $P(x) = 6x^3 - 2x^2 + x + 3$ et le polynôme $D(x) = x^2 - x + 1$. La Division polynomiale de P par D est donnée par :

$$\begin{array}{r|l} 6x^3 - 2x^2 + x + 3 & x^2 - x + 1 \\ - 6x^3 + 6x^2 - 6x & 6x + 4 \\ \hline & 4x^2 - 5x + 3 \\ & - 4x^2 + 4x - 4 \\ \hline & -x - 1 \end{array}$$

Avec cette division, nous obtenons le polynôme quotient $Q(x) = 6x + 4$ et le polynôme reste $R(x) = -x - 1$

$$\Rightarrow P(x) = (x^2 - x + 1)(6x + 4) + (-x - 1)$$

3.2 - Implémentation de la procédure

Mais implémenter la division est difficile non ?

Pas vraiment. Nous verrons ça dans les lignes à suivre.

Commençons par rappeler que le degré du reste R est strictement inférieur à celui du diviseur D . Et tout comme la division des nombre entiers on obtient le reste en soustrayant le produit de Q et de D à P . Nous allons donc représenter le reste comme un vecteur qui a la même taille que la dividende D

Voyons dans le slide suivant, la fonction python qui opère cette division

Code Python de la procédure de la division polynômiale

```
def euclidian_polynomial_division(self, Q: 'P'):  
    """  
        Cette fonction réalise la division polynômiale de P par Q  
        > retourne un tableau des restes et quotients  
    """  
  
    # Si le degré du diviseur dépasse celui de la dividende,  
    # alors le diviseur est le reste  
    if self.degree < Q.degree:  
        return self.coefs.copy(), self.degree + 1  
  
    # création de la dividende : les coefficients du polynôme  
    dividend = self.coefs.copy()  
  
    # création du diviseur : les coefficients du polynôme Q  
    divisor = Q.coefs.copy()  
  
    # initialisation du quotient à une chaîne vide  
    quotients = []
```

Code Python de la division polynômiale (suite)

```
# boucle de calcul du quotient et du reste
for index in range(self.degree, Q.degree - 1, -1):
    # calcul du quotient : division de l'élément à 'index'
    # par le monôme de plus grand degré du diviseur
    # au premier tour cet élément est le monome de plus grand
    # degré de la dividende
    quotient = dividend[index] / divisor[-1]

    # ajout du quotient à la liste des qutients
    quotients.append(quotient)

    # copie et multiplication du diviseur par le quotient
    current_divisor = quotient * divisor.copy()

    # nombre de zéro à gauche pour tendre vers le degré de la divid
    nb_zero_left = index - Q.degree

    # nombre de zéro à gauche pour tendre vers le degré de la divid
    nb_zero_right = self.degree - Q.degree - nb_zero_left
```


Code Python de la division polynômiale (suite)

```
# Ajout des zéros
l = np.zeros(nb_zero_left
r = np.zeros(nb_zero_right)
current_divisor = np.hstack((l, current_divisor, r))

# soustraction et mise à jour de la dividende
dividend -= current_divisor

# la dividende contient à ce niveau les coefficients du
# reste de la division polynômiale

# on inverse l'ordre des coefficients du quotient (ordre croissant)
quotients.reverse()
dividend[Q.degree:] = quotients

# on renvoie (liste de reste, quotient), et la taille de 'reste'
return dividend, Q.degree
```

Appel à la procédure sur P et D

```
# Utilisation de la classe
p = P([3, 1, -2, 6], name="P")
d = P([1, -1, 1], name="D")

# affichage en ligne de commande
print(p)
p.beautiful_display()
print("-----")
print(d)
d.beautiful_display()
print("-----")

# Calcul de la division polynômiale
q_r, degre_r = p.euclidian_polynomial_division(d)

# récupération des coefficients
reste_coefs = q_r[:degre_r]
quotient_coefs = q_r[degre_r:]

# Création et affichage
q = P(quotient_coefs, name="Q")
r = P(reste_coefs, name="R")

print(q)
q.beautiful_display()
print("-----")
print(r)
r.beautiful_display()
```

✓ 0.0s

Appel à la procédure sur P et D (résultat)

$$P(X) = 3.0 + X - 2.0X^2 + 6.0X^3$$

$$P(x) = 6.0x^3 - 2.0x^2 + 1.0x + 3.0$$

$$D(X) = 1.0 - X + X^2$$

$$D(x) = 1.0x^2 - 1.0x + 1.0$$

$$Q(X) = 4.0 + 6.0X$$

$$Q(x) = 6.0x + 4.0$$

$$R(X) = -1.0 - X$$

$$R(x) = -1.0x - 1.0$$

Ces résultats correspondent aux calculs effectués à la sous-section 3.1

Théorème de Sturm

1 - Introduction

2 - Le polynôme et son vecteur

3 - Division polynomiale

4 - Théorème de Sturm

4.1 - Énoncé du théorème

4.2 - Implémentation du théorème

5 - Conclusion

4.1 - Énoncé du théorème

Le théorème de Sturm est un outil puissant en analyse réelle pour déterminer le nombre de racines réelles distinctes d'un polynôme dans un intervalle donné. Ce théorème repose sur la construction d'une suite de Sturm, qui est une suite de polynômes associée à un polynôme donné $P(x)$ et à sa dérivée $P'(x)$.

Suite de Sturm :

- ▶ Soit $P_0(x) = P(x)$ le polynôme initial.
- ▶ Soit $P_1(x) = P'(x)$ la dérivée de $P(x)$.
- ▶ Pour $k \geq 1$, $P_{k+1}(x)$ est défini comme le reste du polynôme obtenu lors de la division euclidienne de $P_{k-1}(x)$ par $P_k(x)$, avec un changement de signe (on multiplie par -).

4.1 - Énoncé du théorème (suite)

Énoncé du théorème :

- ▶ Soit $P(x)$ un polynôme à coefficients réels et soit $\{P_0(x), P_1(x), \dots, P_n(x)\}$ la suite de Sturm associée.
- ▶ Soient a et b deux réels tels que $a < b$.
- ▶ Le nombre de racines réelles distinctes de $P(x)$ dans l'intervalle $[a, b]$ est égal à la différence entre le nombre de changements de signe de la suite de Sturm évaluée en a et en b .

Sur la page suivante, nous présenterons un exemple d'application du **théorème de Sturm**

4.1 - Énoncé du théorème (suite et exemple)

Soit $P(x) = -\frac{x^4}{2} + \frac{x^3}{2} + 4x^2 - x - 6$

- ▶ Étape 1: $P_0(x) = P(x)$.
- ▶ Étape 2: $P_1(x) = P'(x) = -2x^3 + \frac{3x^2}{2} + 8x - 1$
- ▶ Étape 3: Divisions polynômiales successives
 - ▶ Division de $P_0(x)$ par $P_1(x)$

$$\begin{array}{r|l} -\frac{1}{2}x^4 + \frac{1}{2}x^3 + 4x^2 - x - 6 & -2x^3 + \frac{3}{2}x^2 + 8x - 1 \\ \underline{\frac{1}{2}x^4 - \frac{3}{8}x^3 - 2x^2 + \frac{1}{4}x} & \frac{1}{4}x - \frac{1}{16} \\ & \frac{1}{8}x^3 + 2x^2 - \frac{3}{4}x - 6 \\ & \underline{-\frac{1}{8}x^3 + \frac{3}{32}x^2 + \frac{1}{2}x - \frac{1}{16}} \\ & \frac{67}{32}x^2 - \frac{1}{4}x - \frac{97}{16} \end{array}$$

$$\Rightarrow P_2(x) = -R_1(x) = -\frac{67}{32}x^2 + \frac{1}{4}x + \frac{97}{16}$$

4.1 - Énoncé du théorème (suite et exemple)

► Étape 3: Divisions polynômiales successives (Suite)

► Division de $P_1(x)$ par $P_2(x)$

$$\begin{array}{r|l} -2x^3 & + \frac{3}{2}x^2 & + 8x & - 1 \\ 2x^3 & - \frac{16}{67}x^2 & - \frac{388}{67}x & \\ \hline & \frac{169}{134}x^2 & + \frac{148}{67}x & - 1 \\ & - \frac{169}{134}x^2 & + \frac{676}{4489}x & + \frac{16393}{4489} \\ \hline & & \frac{10592}{4489}x & + \frac{11904}{4489} \end{array}$$

$$\Rightarrow P_3(x) = -R_2(x) = -\frac{10592}{4489}x - \frac{11904}{4489}$$

4.1 - Énoncé du théorème (suite et exemple)

- ▶ Étape 3: Divisions polynômiales successives (Suite)
 - ▶ Division de $P_2(x)$ par $P_3(x)$

$$\Rightarrow P_4(x) = -R_3(x) = -\frac{5499025}{1752976}$$

La suite de **Sturm** est donc $\{P_0(x), P_1(x), P_2(x), P_3(x), P_4(x)\}$

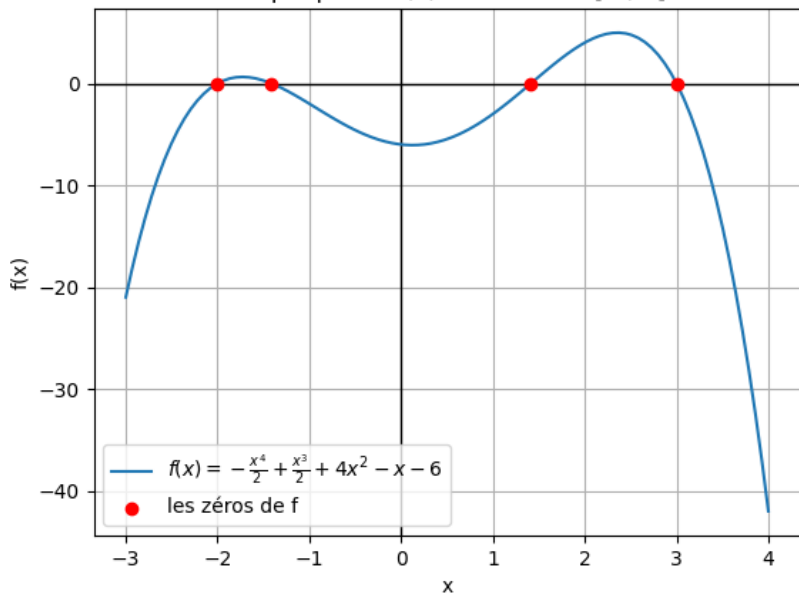
Le tableau de la page suivante présente le nombre de solutions par intervalle $[a, b]$

4.1 - Énoncé du théorème : Nombre de racines

x	$-\infty$	-4	-2	0	2	4	$+\infty$
$P_0(x)$	$-$	$-$	$+$	$-$	$+$	$-$	$-$
$P_1(x)$	$+$	$+$	$+$	$-$	$+$	$-$	$-$
$P_2(x)$	$-$	$-$	$-$	$+$	$-$	$-$	$-$
$P_3(x)$	$+$	$+$	$+$	$-$	$-$	$-$	$-$
$P_4(x)$	$-$	$-$	$-$	$-$	$-$	$-$	$-$
<i>Change</i>	0	1	2	1	1	0	

4.1 - Énoncé du théorème : Tracé la courbe

Graphique de $f(x)$ avec x dans $[-3, 4]$



4.2 - Implémentation du théorème

Tout comme pour la classe P des polynômes, nous allons représenter le processus de **Strum** en Python par une **classe** possédant des attributs et des méthodes.

La classe implémentera et va nous permettre de tester tout ce qu'on a vu ci-dessus.

Nous fournirons ici un code minimaliste de la classe et expliquerons le tout dans le fichier .ipynb associé

Implémentation : La classe **SturmSequence**

```
class SturmSequence:
    """
        Étant donné un polynôme  $P$ , Cette classe génère la séquence
        des fonctions utiles pour la méthode de Sturm
    """
    def __init__(self):
        self.num = -1          # gère les noms successifs des polynômes
        self.sequence = []     # contiendra la séquence de Sturm

    def incr_num(self):
        """ Guide les noms successifs des polynômes à générer """
        self.num += 1

    def generate_name(self):
        """ Guide le nom d'un polynôme """
        self.incr_num()        # choix de nom
        return chr(self.num % 26 + 65)  # le nom sera entre A et Z
```

Implémentation : La classe **SturmSequence** (Suite)

```
def generate_sequence(self, M: P):
    """ Cette fonction génère la séquence de Sturm """
    M.name = self.generate_name()    # Génération du nom
    self.sequence.append(M)          # Ajout à la liste

    if M.degree == 0:                # Si son degré vaut 0, fin
        return

    N = M.derivate(name= self.generate_name()) # La dérivée
    self.sequence.append(N)           # Ajout à la liste

    # Génération du reste des éléments de la séquence
    while N.degree >= 1:
        R = M.get_euclidian_remainder(N) # Reste de la division
        R.name = self.generate_name()    # change nom du reste
        R.oppose_sign()                  # Changement de signe
        self.sequence.append(R)          # Ajout à la liste
        M = N                            # diviseur -> dividende
        N = R                            # reste (* -1) -> diviseur
```

Implémentation : La classe **SturmSequence** (Suite)

```
def number_of_solutions_on(self, a:float, b: float):  
    # a contient le minimum de a et b, b contient le maximum  
    a, b = min(a, b), max(a, b)  
  
    # initatilisatoin des modèle de signes de gauche et de droite  
    left_pattern, right_pattern = "", ""  
  
    for f in self.sequence:  
        # stockage des signes  
        left_pattern += "+" if f.image(a) >= 0 else "-"  
        right_pattern += "+" if f.image(b) >= 0 else "-"  
  
        # comptage des changements de signe aux bornes  
        l_count = left_pattern.count("-+") + left_pattern.count("+ -")  
        r_count = right_pattern.count("-+") + right_pattern.count("+ -")  
    return a, b, np.abs(l_count - r_count)
```

Conclusion

Dans cette présentation, nous avons exploré des aspects de l'application de l'informatique aux mathématiques. Nous avons abordé :

- ▶ La représentation vectorielle des polynômes et les opérations associées
- ▶ L'implémentation de la division polynomiale en utilisant des procédures algorithmiques
- ▶ Le théorème de Sturm et son utilisation pour compter le nombre de racines réelles d'un polynôme dans un intervalle donné

Conclusion (Suite et fin)

Grâce aux outils et techniques présentés, il est possible de résoudre des problèmes complexes de manière plus efficace et précise. Les exemples fournis démontrent l'utilité des concepts théoriques appliqués dans un contexte pratique, notamment dans le cadre de l'enseignement des mathématiques.

En conclusion, l'intégration de l'informatique dans l'étude des mathématiques offre de nombreuses opportunités pour améliorer la compréhension et la résolution des problèmes mathématiques. Les approches numériques et algorithmiques enrichissent les méthodes traditionnelles et ouvrent de nouvelles perspectives pour l'enseignement et la recherche en mathématiques.